МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Балтийский государственный технический университет «ВОЕНМЕХ» им. Д.Ф. Устинова»
(БГТУ «ВОЕНМЕХ» им. Д.Ф. Устинова)

Факультет	Н индекс	Заведующий кафедрой	Н2 индекс кафедры
Выпускающая кафедра	Н2 индекс	Семёнова Е.Г. Фамилия И.О.	подпись
Группа	ЗО712 номер группы	«_____»	_____ 20___ г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА ПО ПРОГРАММЕ БАКАЛАВРИАТА

Кондратовой Екатерины Алексеевны
Фамилия, имя, отчество обучающегося в родительном падеже

На тему Разработка компьютерной игры «Cats Together»

Направление подготовки 09.03.04 (индекс направления) Программная инженерия (полное наименование направления)

Профиль Разработка программно-информационных систем
наименование профиля

Консультант: _____ подпись
если утвержден приказом
_____ ученая степень, ученое звание
Вальштейн К.В. Фамилия ИО
« » _____ 20 ___ г.

Руководитель: _____ подпись
доцент ученая степень, ученое звание
Ракова И.К. Фамилия ИО
« » _____ 20 ___ г.

Обучающийся:
_____ подпись
Кондратова Е.А. Фамилия ИО
« » _____ 20 ___ г.

САНКТ-ПЕТЕРБУРГ
2026 г.

РЕФЕРАТ

СОДЕРЖАНИЕ

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ 6
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ 7
ВВЕДЕНИЕ 8
1 Анализ предметной области 11
1.1 Компьютерные игры с кооперативным игровым процессом 11
1.2 Описание процесса разработки компьютерных игр 13
1.3 Анализ существующих аналогов 16
1.3.1 Цели анализа 16
1.3.2 Fireboy and Watergirl 17
1.3.3 Split Fiction 19
1.3.4 Unravel Two 21
1.3.5 Battleblock Theater 23
1.3.6 Результат обзора аналогов 25
2 Проектирование компьютерной игры «Cats Together» 28
2.1 Описание игры «Cats Together» 28
2.2 Выявление основных вариантов использования 29
2.3 Постановка требований к программному продукту 31
2.3.1 Требования к функциональным характеристикам 31
2.3.2 Требования к входным и выходным данным 34
2.3.3 Требования к надежности 34
2.3.4 Требования к составу и параметрам технических средств 35
2.3.5 Требования к информационной и программной совместимости 35
2.4 Выбор средств разработки 35
2.4.1 Игровой движок 35
2.4.2 Среда разработки 38
2.4.3 Прочие средства разработки 39
2.5 Проектирование архитектуры программного продукта 39
2.5.1 Система управления сценами 40
2.5.2 Система обработки пользовательского ввода 43

2.5.3 Система управления персонажами ... 45
2.5.4 Система управления камерой ... 47
2.5.5 Система взаимодействия с интерактивными объектами ... 48
2.5.6 Система диалогов ... 49
2.5.7 Система пользовательского интерфейса ... 50
2.5.8 Система пользовательских настроек ... 51
2.5.9 Система управления паузой ... 52
2.5.10 Система воспроизведения аудио ... 52
2.5.11 Система локализации ... 53
2.5.12 Система сохранения и загрузки данных ... 54
2.5.13 Интеграция с платформой Steam ... 56
3 Разработка компьютерной игры «Cats Together» ... 57
3.1 Структура проекта и организация процесса разработки ... 57
3.2 Реализация программного продукта ... 57
3.2.1 Игровые сцены и уровни ... 57
3.2.2 Обработка пользовательского ввода ... 57
3.2.3 Управление персонажами ... 57
3.2.4 Управление камерой ... 57
3.2.5 Взаимодействие с интерактивными объектами ... 57
3.2.6 NPC и диалоги ... 57
3.2.7 Управление паузой ... 57
3.2.8 Пользовательский интерфейс ... 57
3.2.9 Пользовательские настройки ... 57
3.2.10 Воспроизведение аудио и интеграция звуковых ресурсов ... 57
3.2.11 Локализация ... 57
3.2.12 Сохранение и загрузка данных ... 57
3.2.13 Интеграция с платформой Steam ... 57
3.3 Тестирование разработанного программного продукта ... 57
ЗАКЛЮЧЕНИЕ ... 58
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ ... 59

ПРИЛОЖЕНИЕ А 62

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящей пояснительной записке применяют следующие термины с соответствующими определениями:

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящей пояснительной записке применяют следующие сокращения и обозначения:

ВВЕДЕНИЕ

В настоящее время компьютерные игры занимают значительное место в сфере цифровых развлечений и являются одним из наиболее активно развивающихся направлений в области информационных технологий. Развитие вычислительных мощностей, графических технологий и сетевой инфраструктуры, а также повышение доступности цифровых платформ для конечных пользователей способствовали росту числа игроков и расширению рынка игровых приложений.

Современные компьютерные игры включают широкий спектр жанров, игровых механик и вариантов взаимодействия игроков. Одной из форм такого взаимодействия являются кооперативные компьютерные игры, предполагающие совместное решение игровых задач несколькими пользователями, что способствует повышению вовлеченности игроков в игровой процесс.

По данным аналитических исследований, интерес к играм с кооперативным режимом продолжает расти. В 2025 году такие игры принесли платформе Steam 8,2 млрд долларов валового дохода, что на 9 % больше по сравнению с 2024 годом и является самым высоким показателем за всю историю платформы. Ранее предполагалось, что после периода пандемии COVID-19, когда совместные игры получили особенно широкое распространение, интерес к кооперативным проектам может снизиться. Однако статистические данные свидетельствуют об обратном: после кратковременного снижения интерес к кооперативным играм вновь начал увеличиваться [1].

Актуальность выбранной темы выпускной квалификационной работы обусловлена необходимостью разработки компьютерной игры «Cats Together» в рамках выполнения заказа индивидуального предпринимателя. Разрабатываемый программный продукт предназначен для распространения на цифровых игровых платформах и использования конечными пользователями.

Объектом исследования являются компьютерные игры с кооперативным игровым процессом.

Предметом исследования является процесс разработки компьютерных игр с кооперативным игровым процессом.

Целью выпускной квалификационной работы является разработка компьютерной игры «Cats Together».

Для достижения поставленной цели необходимо решить следующие задачи:

– рассмотреть понятие и особенности кооперативных компьютерных игр;

– провести анализ процесса разработки компьютерных игр;

– провести сравнительный анализ аналогов разрабатываемого программного продукта;

– сформировать описание игры «Cats Together»;

– выявить основные варианты использования программного продукта;

– сформировать требования к разрабатываемому программному продукту;

– выбрать средства разработки программного продукта;

– выполнить проектирование архитектуры программного продукта;

– определить структуру проекта и организовать процесс разработки;

– реализовать игровые системы программного продукта;

– провести тестирование разработанного программного продукта.

В ходе выполнения работы в качестве методов исследования будут использоваться анализ научной и технической литературы, сравнительный анализ существующих аналогов и системный анализ.

Выпускная квалификационная работа включает введение, три раздела и заключение. В первом разделе выполнен анализ предметной области и

существующих аналогов, а также определены основные аспекты, связанные с разработкой программного продукта. Во втором разделе описан процесс проектирования программного продукта. В третьем разделе представлено описание процесса разработки программного продукта.

1 Анализ предметной области

1.1 Компьютерные игры с кооперативным игровым процессом

Игру можно рассматривать как закрытую формальную систему, в которой действия участников регулируются определенными правилами. В рамках этой системы игроки принимают решения, выполняют действия и взаимодействуют с игровой средой, что изменяет состояние игры и влияет на дальнейший ее ход. Так, существует несколько признаков, позволяющих рассматривать систему как игру:

– наличие конфликта или задачи, в решение которой вовлечен игрок;

– наличие правил, определяющих допустимые действия и ограничения;

– наличие цели и результата, который может быть измерен.

Взаимодействие игрока с правилами, механиками и элементами игровой среды формирует игровой процесс [2].

Компьютерная игра - разновидность игры, представляющая собой компьютерную программу, которая служит для взаимодействия игрока с игровой средой. Программа обрабатывает его действия и реагирует на них, изменяя состояние игрового мира по установленным правилам.

Компьютерные игры могут различаться по количеству участников. Существуют однопользовательские игры, где пользователь взаимодействует только с программной системой, а также многопользовательские игры, в которых несколько пользователей взаимодействуют как с системой, так и друг с другом.

Существует два основных способа реализации многопользовательского взаимодействия игроков: локальный и сетевой. В локальном многопользовательском режиме игроки используют одно устройство, тогда как в случае сетевого взаимодействия организация связи между игроками происходит посредством сети интернет [3].

Одним из ранних примеров многопользовательской компьютерной игры считается Pong, разработанная на основе идей Ральфа Баера для системы Magnavox Odyssey и реализованная компанией Atari в 1972 году в виде аркадного автомата. Данная игра представляет собой простую симуляцию настольного тенниса, в которой два игрока управляют ракетками и отбивают мяч. Несмотря на простоту правил и графики, игра Pong быстро обрела широкую популярность и стала одной из первых коммерчески успешных видеоигр [2].

В многопользовательских играх участники могут соревноваться друг с другом, стремясь получить наилучший результат, или же объединяться и выполнять задачи совместно. Последний формат взаимодействия игроков называется кооперативным игровым процессом.

Одной из ключевых особенностей кооперативного игрового процесса является то, что задачи в нем спроектированы так, что их невозможно выполнить в одиночку: для достижения успеха участники обязаны взаимодействовать друг с другом и координировать свои действия для достижения общей цели. При этом для поддержания вовлеченности участников необходимо организовать правила так, чтобы их вклад в игровой процесс был равнозначен.

Кооперативные игры способствуют усилению социального взаимодействия между игроками. Исследования показывают, что совместная игра помогает повышать уровень коммуникации между участниками, участвует в формировании чувства принадлежности к группе и усиливает ощущение совместного достижения цели. [4].

Такое взаимодействие игроков формирует социальные связи, основанные на совместной деятельности и взаимной поддержке. Кооперативные компьютерные игры способствуют этому, поскольку программная система может четко определить правила, цели и условия выполнения задач, а также предоставить мгновенный отклик на действия игроков, что облегчает понимание структуры совместной деятельности. Кроме

того, цифровая среда позволяет проектировать задачи, выполнение которых требует особой координации действий участников, подталкивая их к сотрудничеству.

1.2 Описание процесса разработки компьютерных игр

Разработка компьютерной игры представляет собой процесс создания программного продукта, объединяющий в себе элементы программной инженерии и творческого проектирования. Несмотря на участие специалистов различных направлений, ключевую роль в построении системы играют программисты, обеспечивающие реализацию и взаимодействие всех подсистем.

Обобщенная модель процесса разработки компьютерной игры может быть представлена в виде функциональной диаграммы IDEF0 верхнего уровня (A0), отражающей основные входы, управляющие воздействия, механизмы и результат процесса. Данная диаграмма приведена на рисунке 1.

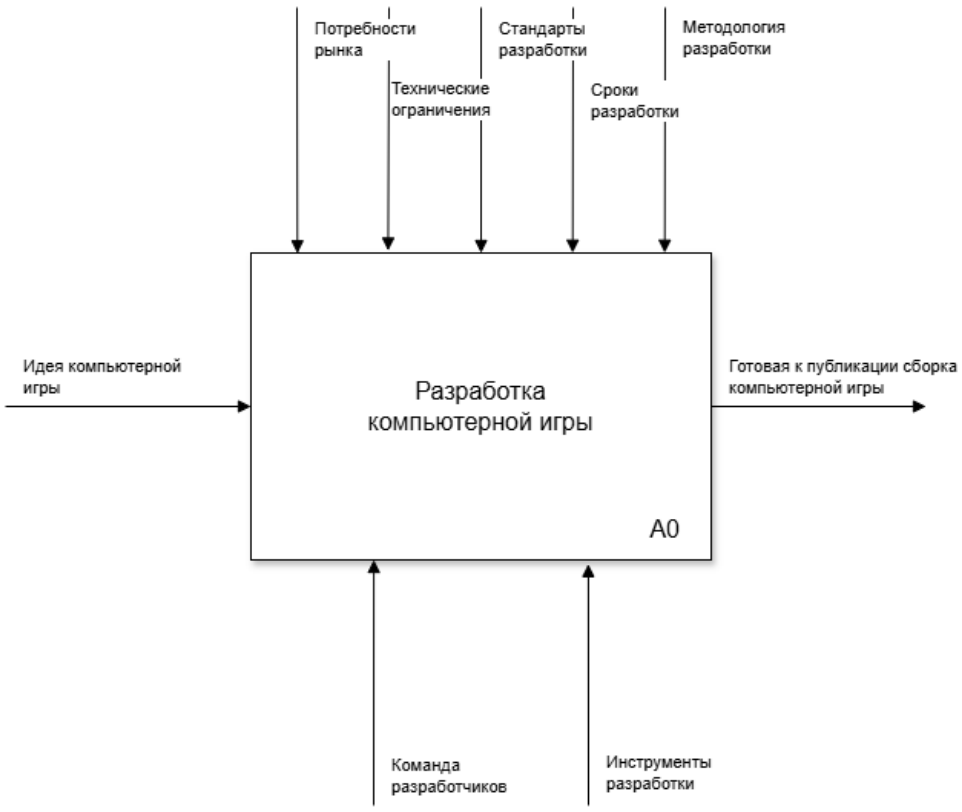

Рисунок 1 – Контекстная диаграмма IDEF0 (A0)

Входом процесса является идея, которая в ходе разработки преобразуется в готовую к публикации сборку компьютерной игры.

Управление процессом осуществляется потребностями рынка, техническими ограничениями, стандартами и методологией разработки, а также сроками выполнения проекта.

В качестве механизмов выступают команда разработчиков и инструменты разработки, обеспечивающие выполнение всех этапов жизненного цикла проекта.

Для более детального рассмотрения процесс разработки может быть декомпозирован на ряд подпроцессов. Данные подпроцессы представлены на диаграмме первого уровня A1-A5, приведенной на рисунке 2.

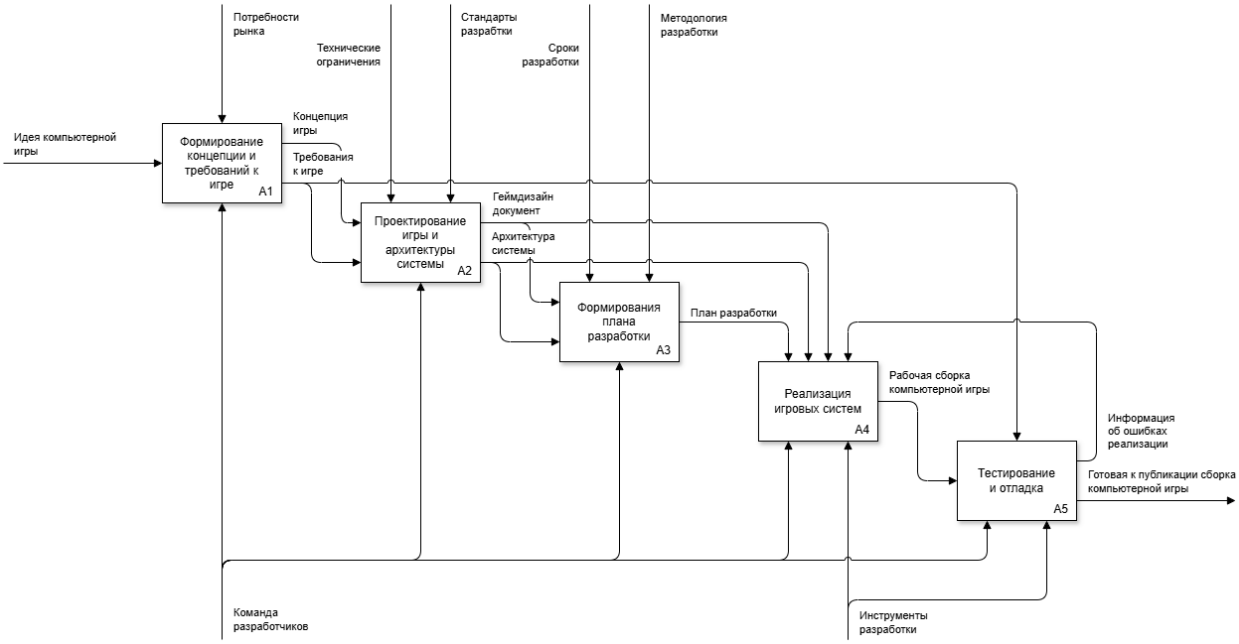

Рисунок 2 – Декомпозиция первого уровня IDEF0 (A1-A5)

На первом этапе процесса разработки выполняется формирование концепции и требований к игре (A1). На данном этапе прорабатывается идея, определяются игровые механики, а также выявляются требования и ограничения. Существенное влияние на данный этап оказывают потребности рынка, определяющие актуальность и направленность проекта.

Далее осуществляется проектирование игры и архитектуры системы (A2). На основе сформированных требований создается гейм-дизайн документ, фиксирующий функциональные и технические характеристики игры, описание игровых систем, логики взаимодействия и структуры данных. На этом этапе также определяется архитектура системы, включая определение

основных компонентов системы и способов их взаимодействия. Проектирование выполняется с учетом технических ограничений и стандартов разработки.

Следующим этапом является формирование плана разработки (A3). На основе гейм-дизайн документа и архитектуры системы производится декомпозиция проекта на задачи, оцениваются трудозатраты и сроки выполнения работ. Планирование осуществляется с учетом выбранной методологии разработки и временных ограничений [5].

После этого выполняется реализация игровых систем (A4), в рамках которой осуществляется программная реализация игровых механик, пользовательского интерфейса, систем управления состоянием игры, а также интеграция внешних библиотек и сервисов. Реализация осуществляется с использованием инструментов разработки и игровых движков [6].

Игровой движок представляет собой программную систему, состоящую из набора взаимосвязанных подсистем, обеспечивающих базовую функциональность игры, включая обработку графики, физики, звука и пользовательского ввода. Использование готовых игровых движков позволяет существенно сократить затраты на разработку за счет повторного использования уже реализованных решений, в то время как создание собственного движка требует значительных временных и финансовых ресурсов [7]–[8].

В процессе реализации осуществляется непрерывная проверка корректности работы отдельных компонентов, включающая анализ логики кода и устранение ошибок непосредственно в ходе разработки. По завершении очередной итерации формируется рабочая сборка игры, которая далее передается на этап тестирования.

На этапе тестирования и отладки (A5) выполняется проверка по методу черного ящика, при которой система рассматривается как единое целое и оценивается с точки зрения пользователя. В результате тестирования

формируется информация об ошибках, которая передается обратно на этап реализации для их устранения.

Таким образом, этапы реализации и тестирования образуют замкнутый цикл, что отражает итеративный характер разработки, при котором сборка последовательно дорабатывается до достижения соответствия установленным требованиям.

В результате разработки формируется протестированная и отлаженная версия компьютерной игры, готовая к публикации. Дальнейшее распространение продукта, включая его размещение на цифровых платформах, осуществляется в рамках отдельного процесса, не входящего непосредственно в этап разработки.

В процессе публикации важную роль играет издатель, который занимается продвижением продукта, маркетингом и организацией распространения. В случае независимой разработки данные функции выполняются непосредственно командой разработчиков.

Таким образом, разработка компьютерной игры представляет собой многоэтапный итеративный процесс, включающий формирование концепции и требований к игре, проектирование игровых систем и их архитектуры, планирование, реализацию и последующее тестирование игровых систем. В результате процесса разработки формируется версия компьютерной игры, соответствующая установленным требованиям и подготовленная для дальнейшего распространения.

1.3 Анализ существующих аналогов

1.3.1 Цели анализа

Анализ существующих аналогов программного продукта проводится с целью выявления подходов к реализации кооперативного игрового процесса, используемых в современных компьютерных играх, а также определения их сильных и слабых сторон.

Сравнительный анализ аналогов будет проводиться по следующим критериям:

– способы взаимодействия игроков в кооперативном режиме;

– степень вовлеченности игроков в процесс прохождения;

– разнообразие игровых механик и ситуаций;

– понятность и предсказуемость игровых механик.

В качестве аналогов программного продукта были выбраны компьютерные игры с кооперативным режимом, в которых реализованы различные подходы к взаимодействию игроков. Различия в реализации кооперативных игровых механик в рассматриваемых аналогах позволит провести сравнительный анализ подходов к организации игрового процесса и определить, какие решения стоит использовать при разработке программного продукта, а также на какие аспекты стоит обратить особое внимание.

1.3.2 Fireboy and Watergirl

Fireboy and Watergirl - серия компьютерных игр в жанре паззл-платформер, созданная независимым разработчиком Oslo Albet. Первая игра серии, Fireboy and Watergirl: The Forest Temple, вышла в 2009 году на игровом портале Armor Games. Игра распространялась как браузерное приложение и работала на технологии Adobe Flash [9].

Игра ориентирована на прохождение двумя игроками: основная идея заключается в совместном решении логических задач на уровне. Такой формат игры подходит для друзей и членов семьи и рассчитан на широкую аудиторию, включая детей и подростков. Проект будет интересен игрокам с низким и средним игровым опытом, поскольку механики в нем довольно простые [10].

Игровой процесс строится вокруг управления двумя персонажами - Fireboy и Watergirl, каждый из которых обладает уникальными способностями. Так, Fireboy может безопасно передвигаться по лаве, тогда как Watergirl - по воде, и каждый из них не может взаимодействовать со стихией другого, что является основной механикой игры. Также на уровнях располагаются различные элементы окружения: кнопки, передвижные платформы, переключатели и прочие механизмы, благодаря которым игроки

открывают новые участки уровня и помогают друг другу преодолевать препятствия [11].

Цель игры - довести обоих персонажей до двери, расположенной в конце уровня. После прохождения каждого уровня игрокам присваивается оценка, зависящая от времени, проведенного за решением головоломки, а также от количества кристаллов, собранных в процессе прохождения. Такая дополнительная механика стимулирует игроков проходить уровень снова и снова для достижения наилучшего результата.

Пример игрового пространства Fireboy and Watergirl представлен на рисунке 3.

Рисунок 3 – Игровое пространство Fireboy and Watergirl

Сюжета в игре нет, а ее структура состоит из отдельных, не связанных между собой уровней. Новые уровни открываются по мере прохождения, а их сложность постепенно увеличивается.

В игре хорошо раскрывается базовый принцип кооперативного взаимодействия. Все механики данной игры направлены на постоянное взаимодействие игроков: они должны согласовывать свои действия и продумывать план перемещения по уровню, чтобы успешно завершить его.

К недостаткам игры можно отнести одинаковую структуру задач на уровне. Игровой процесс развивается только за счет усложнения головоломок, однако в целом остается однообразным, поэтому тяжело оставаться вовлеченным долгое время.

В игре Fireboy and Watergirl показан удачный и простой способ реализации кооперативного игрового процесса, однако данные механики подойдут в основном для базовых головоломок и требуют расширения при разработке более сложных игровых систем.

1.3.3 Split Fiction

Split Fiction - компьютерная игра в жанре приключенческого экшена, разработанная студией Hazelight Studios и изданная 6 марта 2025 года компанией Electronic Arts в рамках программы EA Originals для ПК и игровых консолей [12].

Данная игра рассчитана на двоих игроков, которые управляют персонажами, Мио и Зоуи. Персонажи перемещаются по различным локациям, исследуют новые миры и решают игровые задачи, а ключевой особенностью служит сюжетная составляющая проекта, которая тесно связана с игровым процессом и напрямую влияет на него. Игра ориентирована на широкую аудиторию, но лучше всего подойдет игрокам со средним и высоким игровым опытом, которым интересны приключения и продуманная сюжетная линия. Проект также имеет возрастное ограничение из-за сцен насилия.

В игре присутствует огромное количество разнообразных локаций и диаметрально противоположных миров, которые постоянно сменяют друг друга. Это позволило разработчикам создать большое разнообразие игровых механик и ситуаций, которым игроки продолжают обучаться на протяжении всего игрового процесса, что позволяет им постоянно оставаться

вовлеченными. Также в игре присутствуют короткие дополнительные уровни, косвенно или почти не связанные с сюжетной линией, в которых появляются новые, необычные и иногда даже смешные механики, что вносит еще большее разнообразие в игровой процесс и позволяет игрокам немного отдохнуть [13].

Пример игрового пространства Split Fiction представлен на рисунке 4.

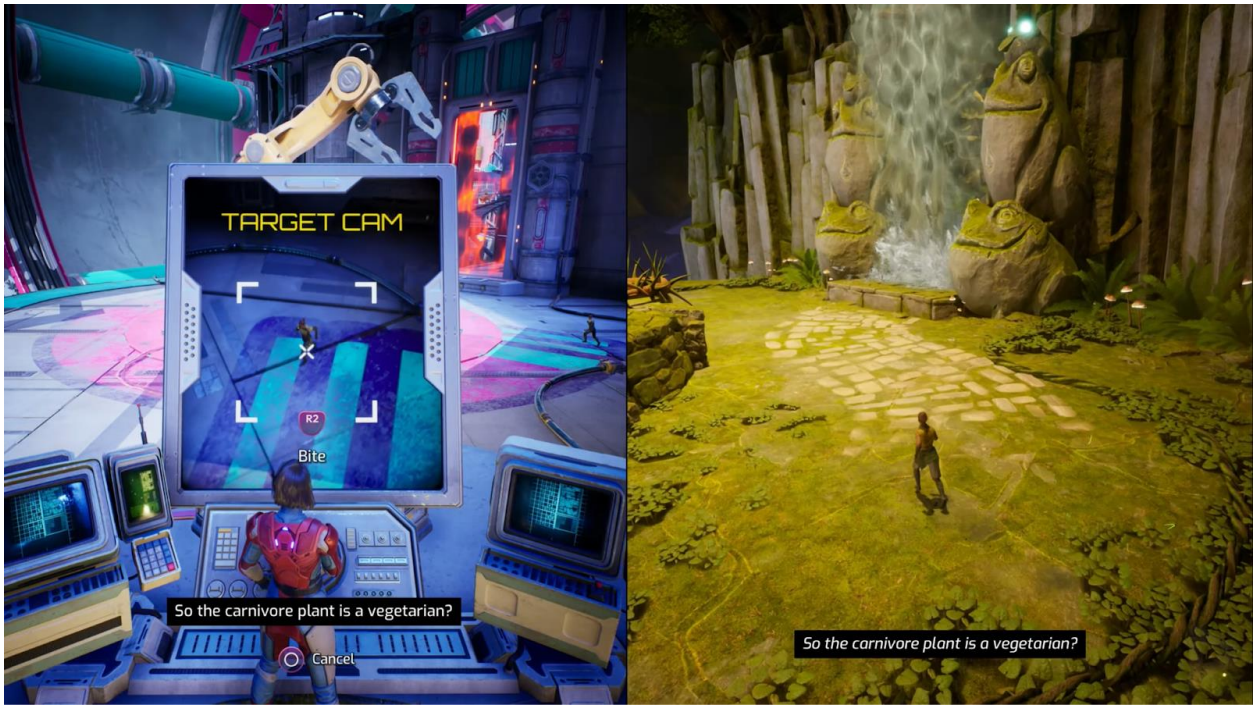

Рисунок 4 – Игровое пространство Split Fiction

Игровой процесс разделен на главы, каждая из которых включает в себя несколько уровней, плавно переходящих друг в друга. Каждый из них представляет собой отдельный мир со своими правилами и способами взаимодействия.

Разработчики продемонстрировали интересный подход к разнообразию игровых ситуаций, создающих вариативность механик. В дополнение к этому, в качестве положительных сторон игры, стоит отметить присутствие дополнительных уровней, напрямую влияющих на вовлеченность игроков в процесс.

К недостаткам можно отнести отсутствие возможности выбора способностей персонажа и их заметную асимметрию. Так, в некоторых эпизодах одному игроку достается более активная и интересная механика,

пока второй выполняет простые, однообразные действия, что негативно сказывается на общем впечатлении от игры.

Также в некоторых ситуациях один игрок может значительно опередить другого. Это происходит из-за того, что не все участки локаций требуют совместных действий, в большей части задач игроки перемещаются отдельно друг от друга. В таком случае, если один из игроков опытнее, возникает заметный дисбаланс: один продвигается значительно быстрее, вынужден ждать, пока партнер его догонит и никак не может ему помочь.

Split Fiction демонстрирует отличный пример взаимосвязи сюжета и игрового процесса, а также поддержания высокого уровня вовлеченности игроков, реализованного с помощью частой смены типа игровой активности. Однако, часть игровых ситуаций допускает прохождение значительных участков уровня в одиночку, в результате чего снижается уровень кооперативного взаимодействия.

1.3.4 Unravel Two

Unravel Two - компьютерная игра в жанре паззл-платформер, разработанная студией Coldwood Interactive и выпущенная компанией Electronic Arts 9 июня 2018 года для ПК и консолей. Данная игра является продолжением серии игр Unravel, однако, в отличие от первой игры, при создании второй части разработчики стремились уделить внимание именно кооперативному взаимодействию [14].

Игра рассчитана на двоих игроков, каждый из которых управляет своим персонажем, называемым Yarny. Отличительной особенностью данной игры является интересная основная механика - нить, связывающая персонажей и позволяющая вместе преодолевать препятствия и решать игровые задачи.

Проект ориентирован на игроков, которым интересны кооперативные платформеры с необычной механикой взаимодействия между персонажами. Игра будет интересна любой возрастной категории игроков, однако подойдет пользователям со средним игровым опытом.

Персонажи могут бегать и прыгать, использовать различные механизмы, однако большинство игровых задач построено именно вокруг основной механики. Персонажи могут раскачиваться, подтягиваться и использовать натяжение нити для передвижения, что требует слаженной работы и точной координации действий игроков. Также в некоторых игровых ситуациях персонажей можно объединить, и тогда управление остается у одного игрока, что позволяет ему помочь своему союзнику преодолеть сложный участок уровня, когда это возможно [15].

Пример игрового пространства Unravel Two показан на рисунке 5.

Рисунок 5 – Игровое пространство Unravel Two

Игровой процесс разделен на уровни, которые, в свою очередь, объединены в главы. После прохождения уровней игрокам становятся доступны дополнительные испытания, требующие более аккуратных и точных действий.

Игра показывает интересный пример механики взаимодействия персонажей, которая напрямую влияет на способ решения игровых задач, что заставляет игроков постоянно оставаться в контакте и учитывать движения партнера, чтобы успешно завершить уровень.

Однако, сама механика нити и ее движения может вводить игроков в заблуждение. Так, например, не всегда понятно, сколько раз нить обернулась вокруг игрового объекта и каков лимит ее натяжения, что усложняет понимание дальнейших действий. Также в игре присутствуют технические проблемы, связанные с физикой этой нити.

При проектировании подобных решений всегда важно учитывать удобство их использования, поскольку если механика работает неочевидно, это может усложнить игровой процесс и причинить игрокам дискомфорт во время прохождения. С другой стороны, такая механика позволяет игрокам постоянно оставаться в контакте, обсуждать свою стратегию, а также поддерживает равное чувство вовлеченности в игровой процесс, что очень важно для кооперативных игр.

1.3.5 Battleblock Theater

BattleBlock Theater - компьютерная игра в жанре паззл-платформер, разработанная студией The Behemoth и изданная этой же студией. Она вышла в 2013 году, сначала релиз состоялся для игровых консолей Xbox, а позже игра стала доступна для ПК [16].

Хотя в игре присутствует одиночный режим, основное внимание уделено именно кооперативной кампании, в которой два игрока проходят уровни вместе и помогают друг другу преодолевать препятствия. Данный проект ориентирован на пользователей, которым интересны платформеры с динамичным игровым процессом, однако из-за демонстрации насилия он имеет возрастное ограничение. Проект подойдет пользователям со средним или высоким игровым опытом, но будет лучше всего, если оба игрока находятся примерно на одном уровне.

Кооперативная кампания основана на прямом взаимодействии между персонажами. В процессе прохождения уровня игроки могут переносить друг друга, подбрасывать на платформы и использовать иные способы взаимодействия друг с другом для достижения общей цели. Так, например,

один персонаж может сесть в лодку, а другой может помочь ему доплыть до противоположного берега. [17].

Игровые локации состоят из блоков, обладающих различными свойствами, что является уникальной чертой BattleBlock Theater. Блоки могут двигаться, исчезать или наносить урон, поэтому игрокам необходимо учитывать их свойства и постоянно использовать эти особенности при прохождении уровня.

Пример игрового пространства BattleBlock Theater представлен на рисунке 6.

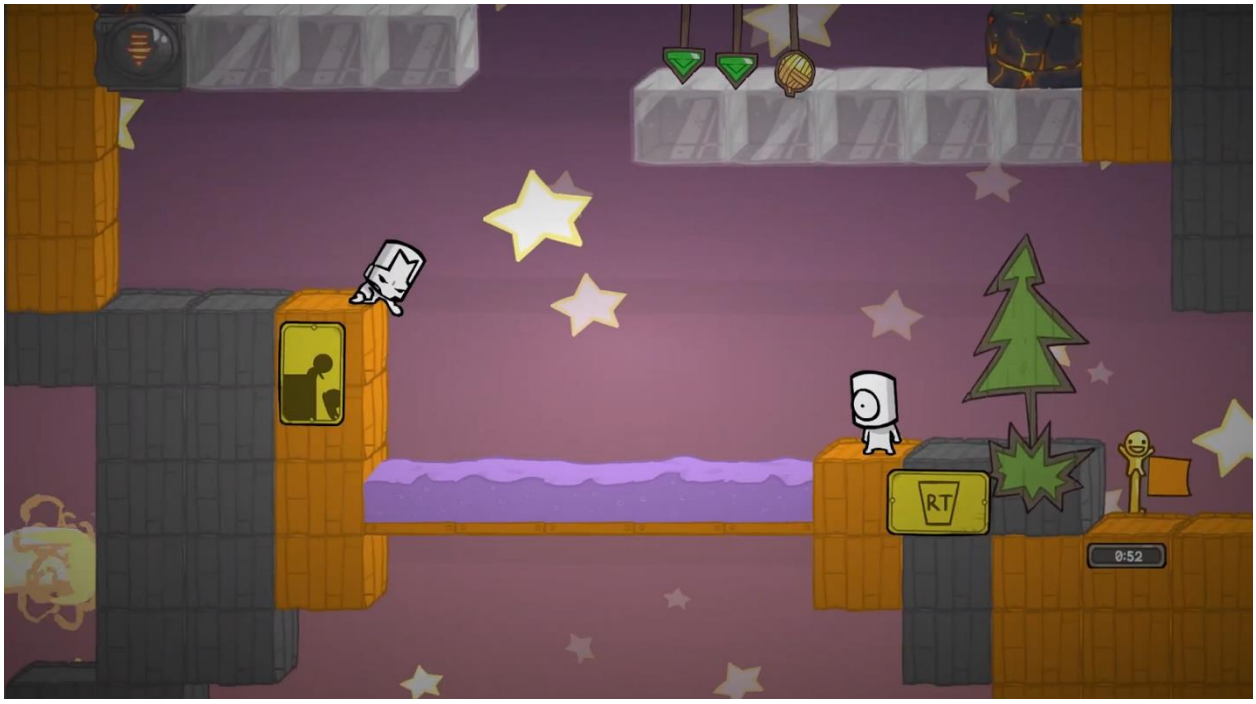

Рисунок 6 – Игровое пространство BattleBlock Theater

Структура игрового процесса состоит из глав, каждая из которых включает несколько независимых друг от друга уровней. Задача игроков - добраться до выхода и уровня, после прохождения которого им присваивается оценка, зависящая от успешности выполнения задач и количества собранных предметов, что повышает интерес к перепрохождению игры.

Достоинствами игры можно считать разнообразие элементов окружения, а также динамичность игрового процесса, обусловленную продуманным дизайном уровней и механикой контактного взаимодействия между персонажами.

К недостаткам BattleBlock Theater можно отнести то, что игроки могут мешать друг другу при прохождении уровня. Некоторые элементы окружения можно перемещать или использовать таким образом, что это создаст проблемы партнеру и затруднит совместное прохождение.

Кроме того, часть уровней допускает прохождение значительных участков локации одним игроком. В такой ситуации второй участник остается в меньшей степени вовлечен в решение задач, что снижает кооперативную составляющую игрового процесса.

В данном проекте некоторые игровые механики не всегда работают корректно в кооперативном режиме, но, тем не менее, игра показывает удачный пример использования контактных взаимодействий между персонажами при проектировании кооперативных игровых задач.

1.3.6 Результат обзора аналогов

На основе проведенного анализа аналогов можно выделить особенности кооперативного взаимодействия, которые важно учитывать при разработке компьютерной игры «Cats Together». Результаты сравнения аналогов программного продукта представлены в таблице 1.

Таблица 1 – Результаты сравнения аналогов программного продукта

Название аналога	Способ взаимодействия игроков	Преимущества игры	Недостатки игры
Fireboy and Watergirl	Одновременная активация механизмов, прохождение препятствий с учетом разных свойств персонажей, передача доступа к элементам уровня	Простое и понятное кооперативное взаимодействие	Повторяемость игровых задач
Split Fiction	Распределение способностей между игроками, последовательное выполнение действий для решения задач, совместное использование механизмов уровня	Разнообразие механик и дополнительные эпизоды	Асимметрия ролей, низкий уровень взаимодействия игроков в некоторых ситуациях
Unravel Two	Совместное перемещение с использованием соединяющей нити, удержание партнера как опоры при преодолении препятствий, создание	Постоянный контакт игроков	Неочевидная работа основной механики

	веревочных мостов, возможность объединить персонажей		
Battleblock Theater (кооперативная кампания)	Помощь партнеру при преодолении препятствий	Динамичное контактное взаимодействие	Игроки могут мешать друг другу, часть задач выполняется одним игроком

В рассмотренных проектах различия в реализации кооперативного игрового процесса проявляются прежде всего в степени взаимодействия игроков между собой, уровне разнообразия игровых ситуаций и понятности используемых механик.

В ряде проектов взаимодействие между игроками является обязательным условием прохождения, что обеспечивает постоянное участие обоих игроков и исключает возможность выполнения задач в одиночку. В других случаях кооперативное взаимодействие носит частичный характер: отдельные игровые ситуации допускают раздельное прохождение, что снижает вовлеченность одного из участников.

Также существенным фактором является разнообразие игровых механик и ситуаций. Проекты с ограниченным набором задач снижают вовлеченность игроков, тогда как регулярное введение новых элементов позволяет поддерживать их вовлеченность на протяжении всего прохождения.

Отдельно следует отметить важность понятности и предсказуемости механик. Сложные или недостаточно наглядные системы взаимодействия могут затруднять восприятие, что негативно влияет на игровой процесс.

Анализ показывает, что наиболее эффективными являются механики, которые:

- требуют участия обоих игроков при решении задач, обеспечивая постоянное взаимодействие между игроками;
- поддерживают разнообразие игровых ситуаций и механик;
- остаются понятными и предсказуемыми для игрока.

При разработке программного продукта данные особенности будут учтены. Игровой процесс будет построен вокруг совместного решения задач и

взаимодействия персонажей с элементами окружения. Основные механики будут направлены на обеспечение постоянного участия обоих игроков и их вовлеченности в игровой процесс. Для поддержания интереса к игре будут реализованы дополнительные уровни, в которых будут использоваться альтернативные игровые ситуации и механики. Это позволит разнообразить игровой процесс и обеспечить поддержание вовлеченности игроков при длительном прохождении.

2 Проектирование компьютерной игры «Cats Together»

2.1 Описание игры «Cats Together»

Разрабатываемая игра «Cats Together» представляет собой 2D-проект в жанре паззл-платформер с поддержкой локального кооперативного режима для двух игроков.

Сюжет игры строится вокруг двух персонажей – котов, которые в результате происшествия выпадают из самолета вместе с грузом посылок. Посылки оказываются разбросаны по различным локациям игрового мира, а сами персонажи находят и заселяются в пустой дом. Основной задачей игроков становится обустройство этого дома и подготовка к празднованию новоселья.

Игровой процесс основан на прохождении уровней, в рамках которых игроки решают логические задачи, взаимодействуют с элементами окружения и знакомятся с неигровыми персонажами. В ходе прохождения персонажи собирают разбросанные по игровому миру посылки, а также выполняют задания, полученные от неигровых персонажей.

С точки зрения механик игра относится к жанру паззл-платформер. Игроки управляют двумя персонажами, способными перемещаться по уровню, взаимодействовать с объектами окружения (кнопками, рычагами, платформами и другими механизмами), а также взаимодействовать друг с другом. В частности, персонажи могут использовать друг друга для преодоления препятствий, например, запрыгивать друг на друга. По мере прохождения также вводятся новые элементы взаимодействия и повышается сложность задач.

Целью прохождения каждого основного уровня является доставка посылки к выходу. Игровая механика предусматривает возможность переноски посылки, при этом персонаж не может выполнять прыжки, удерживая ее. Для преодоления препятствий посылку можно передавать между игроками.

Посылку, полученную в результате прохождения уровня, необходимо распаковать в доме персонажей. Содержимое посылок используется для постепенного обустройства дома на протяжении всего прохождения игры.

Игровой процесс включает как основные, так и дополнительные уровни. Дополнительные (бонусные) уровни открываются после прохождения серии основных уровней и представляют собой отдельные игровые ситуации с уникальными механиками. В рамках таких уровней игроки выполняют задания от неигровых персонажей, включая ловлю рыбы, сбор светлячков, посадку растений и управление воздушным змеем. Данные уровни направлены на разнообразие игрового процесса и поддержание интереса игроков.

Таким образом, игровой процесс строится на сочетании кооперативного взаимодействия, решения логических задач и постепенного усложнения механик, что обеспечивает вовлеченность игроков на протяжении всего прохождения.

2.2 Выявление основных вариантов использования

Для наглядного представления взаимодействия пользователя с программным продуктом была разработана диаграмма вариантов использования (Use Case Diagram).

Диаграмма вариантов использования представляет собой инструмент визуального моделирования, позволяющий описать функциональные возможности системы с точки зрения пользователя и определить границы взаимодействия между актором и программным продуктом. Актором называется внешний участник, взаимодействующий с системой. Данная диаграмма служит основой для последующего проектирования системы и формирования тестовых сценариев.

В контексте данного программного продукта определен один тип актора – игрок, так как все варианты использования реализуются непосредственно через действия пользователя и не предполагают взаимодействия с внешними системами или другими участниками. Несмотря на то, что игра предполагает

участие двух игроков, оба участника представлены на диаграмме в виде единого актора, поскольку их взаимодействие с программным продуктом идентично. Диаграмма вариантов использования для игры «Cats Together» представлена на рисунке 7.

Рисунок 7 – Диаграмма вариантов использования

Игрок может запустить игру, при этом автоматически выполняется загрузка данных сохранения. Перед началом игровой сессии необходимо выбрать слот сохранения, после чего становится доступен игровой процесс. В ходе игрового процесса игрок может проходить уровни, распаковывать посылки и ставить игру на паузу. Настройка игры доступна вне зависимости от текущего состояния игрового процесса. Завершение уровня, распаковка

посылки, выход из игры и изменение настроек сопровождаются автоматическим сохранением данных.

2.3 Постановка требований к программному продукту

Для разработки программного продукта выделены следующие группы требований:

- требования к функциональным характеристикам;
- требования к входным и выходным данным;
- требования к надежности;
- требования к составу и параметрам технических средств;
- требования к информационной и программной совместимости.

Выделенные группы требований служат основой для проектирования и оценки соответствия разрабатываемого программного продукта поставленным задачам.

2.3.1 Требования к функциональным характеристикам

2.3.1.1 Общие функциональные требования

Программный продукт должен обеспечивать:

- отображение графического пользовательского интерфейса игровых и неигровых сцен;
- обработку пользовательского ввода;
- воспроизведение музыкального сопровождения и звуковых эффектов;
- управление игровым процессом в локальном кооперативном режиме с участием двоих игроков;
- сохранение и загрузку данных игрового прогресса и пользовательских настроек.

2.3.1.2 Требования к поддержке сервисов платформы Steam

Программный продукт должен обеспечивать:

- поддержку сервисов цифровой платформы Steam;

– поддержку системы достижений с возможностью их получения в процессе прохождения игры;

– сохранение игрового прогресса с использованием облачного хранилища (Steam Cloud);

– корректную работу оверлея Steam в процессе выполнения программного продукта;

– синхронизацию пользовательских данных при запуске игры на разных устройствах (при наличии соответствующей поддержки платформы).

2.3.1.3 Требования к запуску и навигации

Программный продукт должен обеспечивать:

– предварительную загрузку игровых ресурсов при запуске с последующим отображением главного меню;

– отображение главного меню с возможностью начала игровой сессии, перехода в меню настроек, просмотра титров и завершения работы программного продукта;

– переход между игровыми и неигровыми сценами в соответствии с действиями пользователя и текущим состоянием игрового процесса.

2.3.1.4 Требования к системе сохранений

Программный продукт должен обеспечивать:

– хранение игрового прогресса в нескольких слотах сохранения;

– выбор, установку и использование активного слота сохранения игрового прогресса в рамках текущей игровой сессии;

– загрузку игрового прогресса из выбранного слота сохранения;

– удаление данных слота сохранения игрового прогресса по инициативе пользователя;

– автоматическое сохранение игрового прогресса при завершении уровней и выходе из игровой сессии.

2.3.1.5 Требования к настройкам

Программный продукт должен обеспечивать:

– изменение и сохранение пользовательских настроек, включая параметры звука, графики и языка пользовательского интерфейса;

– настройку управления для каждого игрового персонажа с возможностью выбора схем управления и переназначения пользовательского ввода;

– автоматическое сохранение пользовательских настроек при их изменении.

2.3.1.6 Требования к игровой сессии

Программный продукт должен обеспечивать:

– запуск игровой сессии на основе выбранного слота сохранения игрового прогресса с загрузкой соответствующих данных;

– инициализацию игровой сцены с отображением двух игровых персонажей и передачей управления каждому игроку;

– загрузку обучающего уровня при первом запуске игровой сессии;

– сохранение игрового прогресса при завершении уровней и переходах между сценами.

2.3.1.7 Требования к взаимодействию в игровом процессе

Программный продукт должен обеспечивать:

– взаимодействие игроков с интерактивными объектами игрового окружения с отображением контекстных подсказок при доступности взаимодействия;

– взаимодействие с неигровыми персонажами с отображением диалогов;

– отслеживание выполнения заданий и условий прохождения уровней с соответствующим обновлением состояния игрового процесса.

2.3.1.8 Требования к управлению игровой сессией

Программный продукт должен обеспечивать:

– приостановку игрового процесса (включая приостановку физического моделирования и воспроизведения анимации) с отображением меню паузы по инициативе пользователя;

– доступ из меню паузы к продолжению игровой сессии, изменению настроек, перезапуску текущего уровня и переходу в главное меню.

2.3.2 Требования к входным и выходным данным

Программный продукт должен обеспечивать:

– прием входных данных, формируемых действиями пользователя и передаваемых посредством устройств ввода, включая клавиатуру, мышь и игровой контроллер, а также чтение данных сохранений и пользовательских настроек из локальных файлов формата .dat, представляющих собой JSON-структуру с зашифрованным содержимым.

– формирование выходных данных, включающих визуальную информацию на основе изображений в формате PNG, отображаемую на устройстве вывода, аудиосигнал на основе звуковых файлов в форматах OGG и WAV, воспроизводимый через устройства вывода звука, а также данные игрового прогресса и настроек, записываемые во внешние файлы формата .dat, представляющие собой JSON-структуру с зашифрованным содержимым.

2.3.3 Требования к надежности

Программный продукт должен обеспечивать:

– среднюю частоту кадров не ниже 30 кадров в секунду в игровом процессе;

– обработку пользовательского ввода без ощутимых задержек, обеспечивающую управление в реальном времени;

– корректную обработку ошибок ввода-вывода и ситуаций, связанных с отсутствием или повреждением файлов сохранений и пользовательских настроек.

2.3.4 Требования к составу и параметрам технических средств

Программный продукт должен обеспечивать устойчивую работу при штатной эксплуатации на персональных компьютерах, соответствующих следующим характеристикам:

- 64-битный процессор уровня не ниже Intel Core i5 или аналогичный;
- оперативная память объемом не менее 4 ГБ;
- видеокарта с объемом видеопамяти не менее 4 ГБ.

2.3.5 Требования к информационной и программной совместимости

Программный продукт должен функционировать на персональных компьютерах, работающих под управлением операционных систем Windows и macOS.

2.4 Выбор средств разработки

2.4.1 Игровой движок

Для разработки игры было принято решение использовать готовый игровой движок, поскольку создание собственного требует значительных временных и финансовых затрат, а также существенно усложняет процесс разработки.

Выбор средства разработки осуществлялся с целью определения наиболее подходящего инструмента для разработки компьютерной игры «Cats Together» с учетом требований проекта. Для сравнения были выбраны четыре наиболее распространенных и широко используемых в игровой индустрии решения: Unity, Unreal Engine, Godot Engine и GameMaker.

Сравнительный анализ рассматриваемых средств разработки проводился по следующим критериям:

- лицензирование;
- целевые платформы;
- ориентированность;
- язык программирования;

– объем доступного инструментария.

Unity – кроссплатформенный игровой движок, поддерживающий разработку 2D и 3D проектов для широкого спектра платформ, включая настольные операционные системы, мобильные устройства, веб-платформы и игровые консоли. В качестве языка программирования в данном решении используется C#, что обеспечивает широкие возможности интеграции со сторонними библиотеками и инструментами разработки. Unity обладает широким набором встроенных инструментов, обширной базой готовых решений и одним из крупнейших сообществ разработчиков, что позволяет реализовывать проекты различной степени сложности без существенных ограничений. Для независимых разработчиков предусмотрен бесплатный план Unity Personal, доступный при условии, что годовой доход проекта не превышает 200 тысяч долларов, а при превышении данного порога требуется переход на платный план Unity Pro [18]–[19].

Unreal Engine – игровой движок, разработанный компанией Epic Games и поддерживающий разработку как 2D, так и 3D проектов для настольных операционных систем, мобильных устройств и игровых консолей. Для реализации игровой логики доступно два подхода: программирование на языке C++, а также визуальный скриптинг посредством системы Blueprint, позволяющей выстраивать логику без написания программного кода. Инструментарий данного решения направлен, прежде всего, на разработку трехмерных проектов с высоким качеством графики и включает обширный набор встроенных инструментов и расширений. Данный игровой движок бесплатен при условии, что годовая выручка компании не превышает 1 миллиона долларов, после чего требуется приобретение платной подписки Unreal Subscription [20].

Godot Engine – игровой движок с открытым исходным кодом, поддерживающий разработку 2D и 3D проектов для настольных операционных систем, мобильных устройств и веб-платформ. Для разработки возможно использование нескольких языков программирования: встроенного

языка GDScript, а также C# и C++. Рабочий процесс в Godot Engine основан на собственной архитектуре сцен и узлов, которая существенно отличается от подходов, принятых в других широко распространенных средствах разработки игр, что накладывает определенные ограничения на доступный инструментарий и возможности расширения проекта. Движок распространяется полностью бесплатно под лицензией MIT, не накладывающей никаких ограничений по доходу или условиям использования [21].

GameMaker – специализированное средство разработки, ориентированное прежде всего на создание 2D-игр для широкого спектра платформ, включая настольные операционные системы, мобильные устройства, веб-платформы и игровые консоли. В качестве языка программирования используется собственный язык GML, допускающий как визуальное программирование посредством блоков GML Visual, так и написание кода вручную. Инструментарий данного решения оптимизирован для 2D-разработки и располагает развитой библиотекой готовых ассетов и расширений, однако возможности масштабирования и гибкость настройки проекта существенно ограничены. Для некоммерческих проектов GameMaker распространяется бесплатно, а для коммерческого использования или публикации на консолях требуется приобретение платной лицензии [22].

Результаты сравнения игровых движков представлены в таблице 2.

Таблица 2 – Результаты сравнения игровых движков

Критерий	Unity	Unreal Engine	Godot Engine	Game Maker
Лицензирование	Бесплатно до 200 тысяч долларов дохода от проекта	Бесплатно до 1 миллиона долларов выручки компании	Бесплатно (MIT)	Бесплатно для некоммерческих проектов
Целевые платформы	Настольные ОС, мобильные устройства, веб-платформы, игровые консоли	Настольные ОС, мобильные устройства, игровые консоли	Настольные ОС, мобильные устройства, веб-платформы	Настольные ОС, мобильные устройства, веб-платформы, игровые консоли

Область применения	2D и 3D	Преимущественно 3D	2D и 3D	Преимущественно 2D
Язык программирования	C#	C++, Blueprint	GDScript, C#, C++	GML
Объем доступного инструментария	Обширный	Обширный	Средний	Ограниченный

Анализ рассмотренных игровых движков показал, что каждый из них имеет свои особенности и ограничения применительно к требованиям проекта. Unreal Engine ориентирован на трехмерные проекты и избыточен для 2D разработки. GameMaker специализирован на 2D, однако ограничен в масштабировании и интеграции со сторонними сервисами. Godot Engine располагает необходимым инструментарием, однако его архитектура и экосистема менее развиты по сравнению с Unity. Unity, в свою очередь, сочетает полноценную поддержку 2D разработки с широкими возможностями интеграции со сторонними библиотеками и сервисами.

Исходя из результатов сравнительного анализа, с учетом требований проекта, для разработки игры «Cats Together» был выбран игровой движок Unity.

2.4.2 Среда разработки

Помимо игрового движка, для написания программного кода необходима интегрированная среда разработки. В рамках выбора подходящей среды разработки будут рассмотрены два решения: Microsoft Visual Studio и JetBrains Rider.

Microsoft Visual Studio – комплексная интегрированная среда разработки от компании Microsoft, поддерживающая разработку на языке C# и широко используемая совместно с Unity. Интеграция с Unity реализуется посредством официального расширения. Полнофункциональная версия Visual Studio распространяется на платной основе, тогда как бесплатная редакция Community имеет ограничения по условиям коммерческого использования [23].

JetBrains Rider – кроссплатформенная интегрированная среда разработки от компании JetBrains, ориентированная на работу с языком C# и платформой .NET. Среда обеспечивает нативную интеграцию с Unity без необходимости установки дополнительных расширений, предоставляет автодополнение с учетом Unity API и поддерживает встроенную отладку непосредственно из среды разработки. Для студентов и преподавателей JetBrains Rider доступен бесплатно, однако в рамках коммерческого проекта требуется приобретение платной подписки [24]–[25].

Для написания программного кода была выбрана среда разработки JetBrains Rider. Несмотря на платное распространение в рамках коммерческого проекта, данное решение обеспечивает удобную интеграцию с Unity и не требует дополнительной настройки для полноценной работы с проектом.

2.4.3 Прочие средства разработки

Визуальные ресурсы игры – спрайты, фоны и элементы пользовательского интерфейса – разрабатывались специалистами в области графического дизайна. Звуковое сопровождение, включая музыку и звуковые эффекты, создавалось звукорежиссером. Выбор инструментов для создания данных ресурсов находится в компетенции соответствующих специалистов и не рассматривается в рамках настоящей выпускной квалификационной работы.

2.5 Проектирование архитектуры программного продукта

Архитектура программного продукта построена с учетом компонентно-ориентированного подхода Unity, при котором игровой объект представляет собой контейнер, поведение которого формируется набором прикрепленных компонентов. Такой подход обеспечивает модульность и возможность повторного использования функциональности без изменения структуры объекта.

Игровые объекты размещаются на сценах, каждая из которых представляет собой отдельную логическую единицу приложения, например, главное меню, игровой уровень или вспомогательные сцены интерфейса.

Разделение на сцены обеспечивает изоляцию ресурсов и упрощает управление состоянием приложения.

На этапе проектирования логика приложения была разделена на отдельные подсистемы по функциональному назначению. Такой подход снижает связанность компонентов, упрощает сопровождение проекта и делает архитектуру более удобной для последующего расширения. С учетом выделенных функциональных требований к программному продукту были предусмотрены следующие подсистемы:

- система управления сценами;
- система обработки пользовательского ввода;
- система управления персонажами;
- система управления камерой;
- система взаимодействия с интерактивными объектами;
- система диалогов;
- система управления паузой;
- система пользовательского интерфейса;
- система пользовательских настроек;
- система воспроизведения аудио;
- система локализации;
- система сохранения и загрузки данных.

Для каждой из выделенных подсистем необходимо установить зону ответственности, описать выполняемые ею функции и определить принципы внутренней организации системы.

2.5.1 Система управления сценами

В игровом движке Unity основной структурной единицей проекта является сцена, представляющая собой отдельный файл, содержащий игровой мир со своим набором объектов, скриптов и настроек. В рамках

проектирования системы управления сценами для игры «Cats Together» были выделены следующие виды сцен:

- сцена инициализации;
- сцена «Главное меню»;
- сцена «Дом»;
- сцена «Уровень»;
- сцена «Карта уровней»;
- сцена «Титры».

На основе выделенных видов сцен, для наглядного представления логики навигации между ними, был построен граф переходов, представленный на рисунке 8.

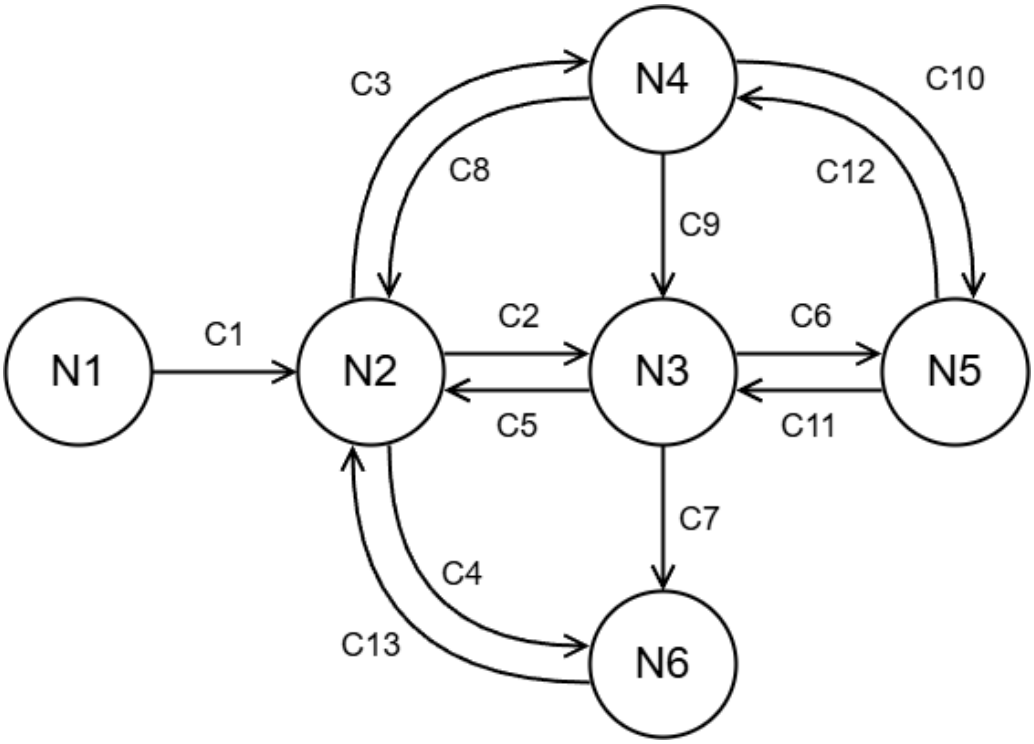

Рисунок 8 – Граф переходов между сценами

На приведенном графе вершинами обозначены сцены приложения, а соединяющие их ребра отражают возможные переходы. Подробное описание каждого перехода с указанием условий его осуществления приведено в таблице 3.

Таблица 3 – Описание переходов между сценами

Вершина	Условие перехода	Следующая вершина
N1 – Сцена инициализации	C1. Завершена предварительная загрузка ресурсов	N2 – Сцена «Главное меню»
N2 – Сцена «Главное меню»	C2. Выбран слот сохранения при условии, что обучающий уровень пройден	N3 – Сцена «Дом»
	C3. Выбран слот сохранения при условии, что обучающий уровень не пройден	N4 – Сцена «Уровень»
	C4. Нажата кнопка «Титры»	N6 – Сцена «Титры»
N3 – Сцена «Дом»	C5. Нажата кнопка «Выход в меню»	N2 – Сцена «Главное меню»
	C6. Переход через объект «Ворота»	N5 – Сцена «Карта уровней»
	C7. Игра завершена	N6 – Сцена «Титры»
N4 – Сцена «Уровень»	C8. Нажата кнопка «Выход в меню»	N2 – Сцена «Главное меню»
	C9. Выполнены условия завершения уровня и совершен переход через объект «Ворота», если это обучающий, последний или бонусный уровень	N3 – Сцена «Дом»
	C10. Выполнены условия завершения уровня и совершен переход через объект «Ворота», если это обычный уровень	N5 – Сцена «Карта уровней»
N5 – Сцена «Карта уровней»	C11. Нажата кнопка «Дом»	N3 – Сцена «Дом»
	C12. Нажата кнопка уровня	N4 – Сцена «Уровень»
N6 – Сцена «Титры»	C13. Завершен показ титров	N2 – Сцена «Главное меню»

Сцена инициализации должна отвечать за проверку лицензии Steam и предварительную загрузку ресурсов, необходимых для работы приложения.

После загрузки ресурсов должен осуществляться переход на сцену «Главное меню», обеспечивающую точку входа в игровой процесс. На сцене «Главное меню» игроку должны быть доступны настройки приложения, переход к просмотру титров, а также выбор слота сохранения для начала или продолжения игры.

После выбора слота сохранения должен осуществляться переход на сцену «Уровень» (обучающий уровень) или на сцену «Дом» в зависимости от того, был ли пройден обучающий уровень ранее. Сцена «Дом» является основной сценой, в которую игрок должен возвращаться на протяжении всей игры после прохождения уровней. В ней должна происходить распаковка

посылок и взаимодействие с неигровыми персонажами. Также из этой сцены должен быть доступен переход к выбору следующего уровня – на сцену «Карта уровней». Сцена «Карта уровней» должна предоставлять навигацию по доступным уровням и позволять вернуться в сцену «Дом».

Сцена «Уровень» объединяет в себе все игровые уровни, поскольку их общая логика едина – игрок должен получить предмет и вынести его через объект «Ворота». При этом уровни подразделяются на три вида – обучающий, обычный и бонусный. Уровни должны проходиться в определенном порядке, поэтому для их хранения и загрузки по номеру необходимо предусмотреть реестр уровней.

По завершении игры должен осуществляться переход на сцену «Титры». По окончании показа титров должен происходить переход на сцену «Главное меню».

Для реализации описанной логики навигации необходимо разработать систему управления сценами. Встроенный в Unity класс SceneManager предоставляет базовые операции загрузки и выгрузки сцен, однако логика переходов между сценами должна быть централизована. Для этого необходимо предусмотреть класс-обертку, являющийся единой точкой входа для всех переходов в приложении и исключающий прямое обращение к SceneManager из игровых объектов. Класс-обертка должен предоставлять набор методов, соответствующих переходам, описанным в таблице 3, и инкапсулировать логику их выполнения. Помимо этого, частью системы управления сценами является реестр уровней, обеспечивающий хранение и загрузку уровней по номеру.

2.5.2 Система обработки пользовательского ввода

Система обработки пользовательского ввода отвечает за получение сигналов от устройств ввода, их интерпретацию и передачу соответствующих команд игровым объектам. Ключевым требованием к данной системе является абстрагирование игровой логики от конкретных устройств ввода. Также система должна поддерживать несколько схем управления одновременно,

обеспечивая возможность игры как с клавиатуры, так и с геймпада без изменения логики игровых объектов.

Поскольку разрабатываемая игра должна поддерживать режим локального кооператива, для обеспечения одновременного управления двумя игроками были выделены четыре схемы управления:

- клавиши W, A, S, D;
- клавиши-стрелки;
- игровой контроллер 1;
- игровой контроллер 2.

Каждый игрок должен иметь возможность выбрать одну из доступных схем, при этом два игрока не могут использовать одну и ту же схему одновременно. Выбранная схема управления описывает полный набор действий для игрока и закрепляется за его персонажем на протяжении всего игрового процесса. Система должна обеспечивать корректную обработку подключения и отключения устройств ввода. В качестве резервного способа управления всегда должна выступать клавиатура, поэтому при непредвиденном отключении игрового контроллера игроку должна автоматически назначаться свободная схема управления с клавиатуры. Если после отключения контроллера игрок не менял схему управления вручную и тот же контроллер был подключен повторно, система должна автоматически восстановить привязку данного контроллера к игроку.

Для каждой схемы управления были определены следующие игровые действия, абстрагированные от конкретных устройств ввода:

- движение;
- взаимодействие;
- увеличение обзора камеры;
- пауза.

Действие «движение» должно формировать вектор направления перемещения персонажа на основе четырех направлений. Действие

«взаимодействие» должно активировать взаимодействие с доступными игровыми объектами. Действие «увеличение обзора камеры» должно временно расширять область видимости. Действие «пауза» должно приостанавливать игровой процесс и открывать меню паузы.

Помимо игрового контекста, система ввода должна обслуживать навигацию по пользовательскому интерфейсу. Для этого необходимо предусмотреть отдельную карту действий, в которой клавиши перемещения каждой схемы используются для навигации между элементами меню, а действие «взаимодействие» выполняет выбор активного элемента. Навигация по интерфейсу также должна поддерживать управление мышью.

Также, для схем управления с клавиатуры, игрок должен иметь возможность переназначить привязку любого действия к удобному для него элементу ввода. Измененные привязки должны сохраняться между сессиями и применяться при следующем запуске приложения.

Для централизованного управления описанной логикой необходимо разработать компонент-менеджер ввода, отвечающий за назначение схем управления игрокам и отслеживание состояния подключенных устройств.

2.5.3 Система управления персонажами

Система управления персонажами отвечает за поведение игровых персонажей на протяжении всей игровой сессии. В зону ответственности данной системы входят физическое перемещение персонажей, обнаружение столкновений с окружением, воспроизведение анимации и обработка пользовательского ввода. Каждая из перечисленных ответственностей должна быть реализована в виде независимого компонента, взаимодействующего с остальными компонентами через единый интерфейс персонажа.

Для проектирования системы управления персонажами был выбран паттерн «Конечный автомат». В рамках данного паттерна объект в каждый момент времени находится ровно в одном состоянии и переходит между ними по явно определенным условиям. Для наглядного представления логики переходов между основными состояниями была разработана UML-диаграмма

состояний персонажа. Диаграмма состояний является инструментом моделирования поведения объекта и отображает множество возможных состояний, условия переходов между ними, а также начальное состояние. На рисунке 9 представлена диаграмма, отражающая основные состояния персонажа и переходы между ними.

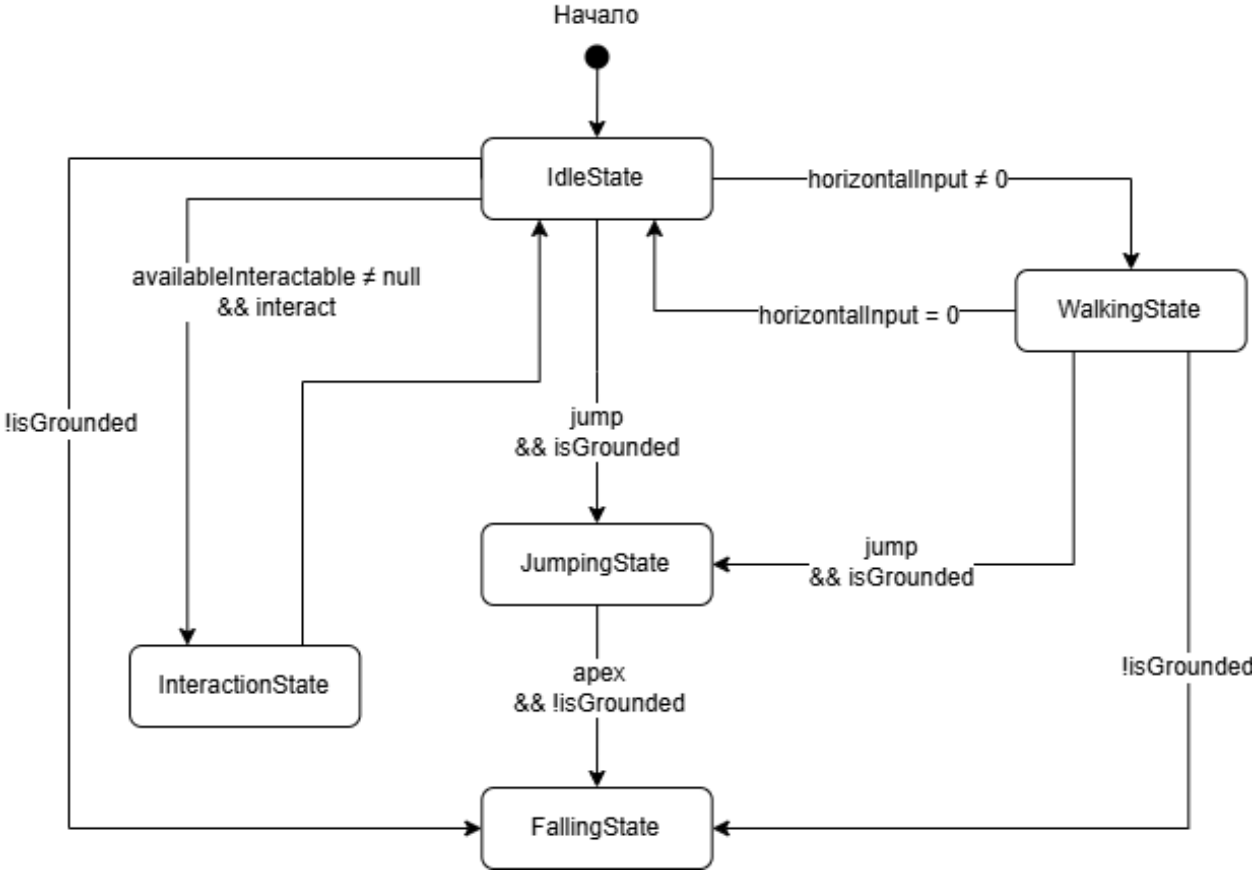

Рисунок 9 – Диаграмма состояний персонажа

Начальным состоянием персонажа является состояние покоя. Переход в состояние ходьбы должен осуществляться при наличии горизонтального ввода, обратный – при его отсутствии. Переход в состояние прыжка должен осуществляться из обоих состояний при получении команды прыжка и нахождении персонажа на земле. По достижении вершины траектории должен осуществляться переход в состояние падения, из которого персонаж должен вернуться в состояние покоя при приземлении. Переход в состояние взаимодействия должен происходить только из состояния покоя при наличии доступного объекта. По завершении взаимодействия персонаж должен вернуться в состояние покоя. Конечное состояние не предусмотрено,

поскольку персонаж находится в одном из состояний на протяжении всей игровой сессии.

Поскольку игра предполагает одновременное присутствие двух управляемых персонажей в сцене, система должна обеспечивать их независимую работу. Каждому персонажу должен предоставляться собственный экземпляр конечного автомата, а также отдельные компоненты, отвечающие за обработку физики и анимации. Взаимодействие с системой обработки пользовательского ввода должно осуществляться посредством подписки на события, при которой появлении персонажа в сцене ему назначается схема управления в соответствии с сохраненными настройками игрока. Сигналы данной схемы должны поступать в компонент обработки ввода персонажа и инициировать соответствующие переходы между состояниями конечного автомата.

2.5.4 Система управления камерой

Система управления камерой отвечает за динамическое позиционирование и масштабирование игрового вида на протяжении всей игровой сессии. Ее основной задачей является обеспечение видимости обоих игровых персонажей вне зависимости от их расположения на сцене, а также обработка команд масштабирования со стороны игрока.

Поскольку поведение камеры сводится к двум четко разграниченным режимам работы с явными условиями переходов между ними, для ее проектирования был применен паттерн «Конечный автомат» – аналогично решению, принятому при проектировании системы управления персонажами.

Начальным является состояние следования за персонажами. В нем камера должна на каждом кадре вычислять центральную точку между двумя персонажами, плавно перемещаться к ней и корректировать масштаб так, чтобы оба персонажа оставались в пределах видимой области. При получении от игрока команды увеличения должен осуществляться переход в состояние масштабирования, в котором поле зрения камеры увеличивается до заданного максимального значения, а возможность управления персонажами

блокируется на все время нахождения в данном состоянии. По завершении пользовательского ввода камера должна возвращаться в состояние следования, после чего управление персонажами должно быть восстановлено.

2.5.5 Система взаимодействия с интерактивными объектами

В разрабатываемой игре предусмотрен ряд объектов окружения, с которыми игроки могут взаимодействовать в игровом процессе. В зависимости от характера взаимодействия можно выделить следующие виды таких объектов:

- интерактивные объекты;
- переносимые объекты;
- собираемые объекты;
- объекты-получатели;
- объекты, инициирующие диалог.

Все перечисленные объекты являются подвидом интерактивных, но под интерактивными объектами как отдельным видом понимаются те, которые по команде взаимодействия просто выполняют заложенное в них действие, например, открывают проход на уровне. Переносимые объекты игрок может поднять, нести с собой и положить в произвольном месте. Собираемые объекты являются подвидом переносимых, но, помимо переноса, их можно положить в объект-получатель, если тип предмета в руках совпадает с тем, который ожидает объект-получатель. Объекты, инициирующие диалог, представлены в контексте разрабатываемой игры неигровыми персонажами.

Для наглядного представления организации перечисленных видов объектов была разработана UML-диаграмма классов, отображающая классы системы взаимодействия, их методы и отношения между ними. Диаграмма классов системы взаимодействия с интерактивными объектами представлена на рисунке 10.

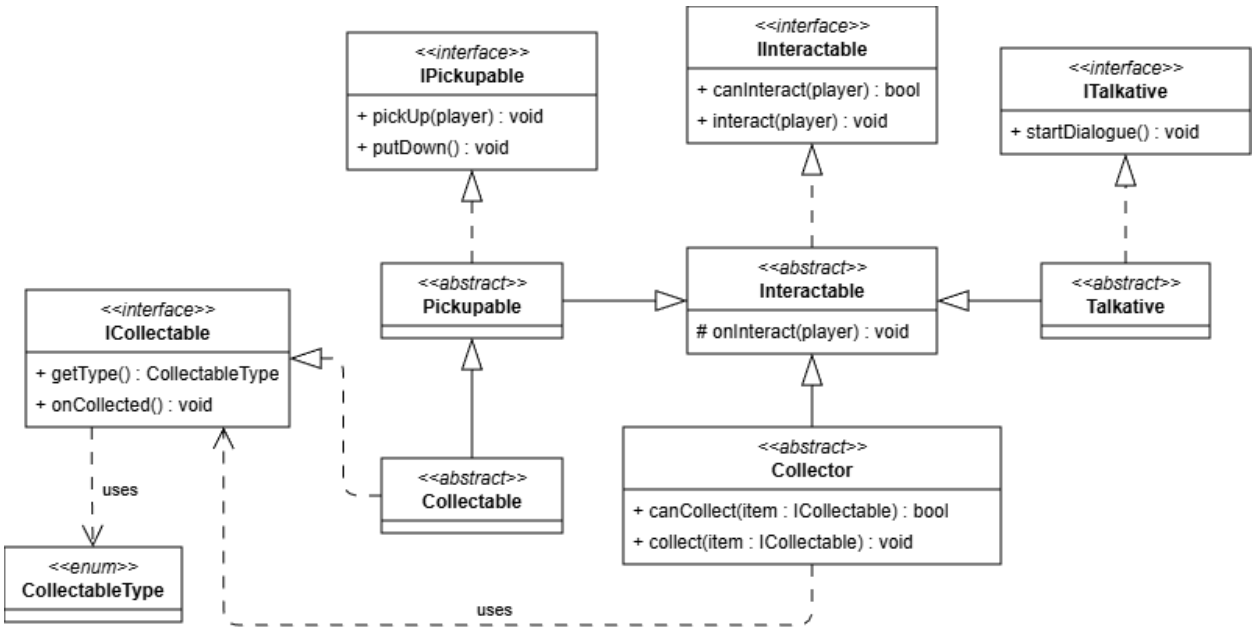

Рисунок 10 – Диаграмма классов системы взаимодействия

Базовым элементом иерархии является абстрактный класс Interactable, реализующий интерфейс IInteractable и выступающий родительским для всех перечисленных видов объектов. Метод interact реализован по паттерну «Шаблонный метод» и делегирует управление защищенному методу onInteract, определяемому в подклассах. Переносимые объекты строятся на основе абстрактного класса Pickupable, расширяющего Interactable и реализующего интерфейс IPickupable. Собираемые объекты представлены абстрактным классом Collectable, расширяющим Pickupable и реализующим интерфейс ICollectable, который дополнительно декларирует тип объекта через перечисление ItemType. Абстрактные классы Collector и Talkative расширяют Interactable независимо, реализуя поведение объектов-получателей и объектов, инициирующих диалог, соответственно.

При приближении игрока к интерактивному объекту система должна определить, доступно ли взаимодействие с ним и, если взаимодействие доступно, должно появиться визуальное обозначение возможности взаимодействия.

2.5.6 Система диалогов

Система диалогов предназначена для воспроизведения текстовых реплик неигровых персонажей. Диалоговые данные должны храниться во

внешнем источнике и загружаться во время выполнения по строковому ключу. Правки текста должны вноситься напрямую в файл данных, не затрагивая программный код.

Каждая реплика должна иметь идентификатор диалога и порядковый индекс внутри последовательности. Помимо текста, к каждой реплике должны быть привязаны события, публикуемые в момент ее начала и завершения, а также отдельное событие по завершении всей последовательности. Данный механизм обеспечивает синхронизацию нарративных триггеров с игровыми событиями без жесткой привязки логики к конкретному тексту.

Диалог в рамках данного проектного решения представляет собой монолог – последовательность реплик без ветвления и выбора ответов, что упрощает структуру данных и снижает сложность управления состоянием системы. Каждый монолог должен иметь флаг повторяемости, определяющий, может ли последовательность быть воспроизведена повторно после завершения.

Запуск диалога должен осуществляться через выделенный компонент, размещаемый на игровом объекте, и поддерживать два режима активации – по явному взаимодействию игрока с объектом и по триггеру при входе персонажа в заданную область. В обоих случаях компонент должен передать строковый ключ в систему диалогов, которая запросит соответствующую последовательность реплик и начнет ее воспроизведение.

2.5.7 Система пользовательского интерфейса

Пользовательский интерфейс приложения должен быть организован в виде иерархической структуры экранов с главным меню в качестве корневого узла.

Главное меню должно предоставлять доступ к четырем основным действиям. Переход к игровой сессии должен осуществляться через экран выбора слота сохранения, на котором отображаются три слота с датой последнего сеанса - каждый слот должен поддерживать очистку и загрузку сохраненного состояния. Из экрана выбора слота должен быть предусмотрен

возврат в главное меню. Экран настроек должен быть организован в виде трех подразделов - общих параметров, схем управления и переназначения клавиш - с возможностью возврата в главное меню. Экран титров должен быть доступен как из главного меню, так и по завершении прохождения игры.

Меню паузы должно быть доступно в любой момент игровой сессии и предоставлять быстрый доступ к настройкам звука, переход к полному экрану настроек и возврат в главное меню. На игровых уровнях меню паузы должно дополнительно отображать номер текущего уровня и содержать кнопку перезапуска. На сцене «Дом» также должна быть доступно меню паузы, однако перезапуск сцены должен быть недоступен, поскольку данная сцена не является уровнем.

2.5.8 Система пользовательских настроек

Система пользовательских настроек предназначена для управления параметрами, определяющими поведение и внешний вид приложения в соответствии с предпочтениями игрока. Сохранения и загрузка настроек должны осуществляться через систему сохранения, а при отсутствии сохраненных данных должны применяться значения по умолчанию.

Система должна обеспечить для игрока возможность изменения следующих параметров:

- громкость музыки;
- громкость звуковых эффектов;
- разрешение экрана;
- режим экрана (полный экран, оконный);
- вертикальная синхронизация кадров;
- язык интерфейса;
- схема управления для каждого из двух игроков;
- переназначение клавиш управления для каждой их схем управления с клавиатуры;

Параметры звука, графики и языка интерфейса должны применяться мгновенно при их изменении. Назначение схем управления должно осуществляться посредством диалога, при переходе в режим изменения схемы управления игрок должен выбрать схему ввода, после чего должен выбрать персонажа, к которому будет привязана данная схема, после чего данный выбор будет необходимо повторить для второго персонажа. Назначение переопределения клавиш для схем ввода должно осуществляться по следующему сценарию: после нажатия кнопки перепривязки система должна перейти в режим ожидания и связать следующую нажатую клавишу с соответствующим действием.

Каждый параметр должен управляться отдельным компонентом, связанным с соответствующим элементом интерфейса. При изменении значения компонент должен применить его в системе и передать обновленные данные в систему сохранения.

Для применения сохраненных настроек игрока при новом запуске приложения должны быть разработанные отдельные компоненты, получающие сохраненные данные и применяющие их в системе при старте приложения.

2.5.9 Система управления паузой

Система управления паузой должна обеспечивать приостановку и возобновление игрового процесса по инициативе игрока. Центральным элементом системы должен являться статический класс, хранящий текущее состояние паузы и публикующий событие при его изменении.

Во время постановки игры на паузу должно быть приостановлено физическое моделирование, воспроизведение игровых анимаций, а также выполнение функций обновления игровой логики и иных процессов, выполняемых в течение игрового времени.

2.5.10 Система воспроизведения аудио

Для воспроизведения звука в Unity используются компоненты источников звука, размещаемые на игровых объектах сцены. Каждому

источнику звука можно присвоить свою группу в микшере звука, что позволяет в одном месте управлять параметрами нескольких источников.

Источники звука предусматривают несколько режимов воспроизведения в пространстве – с учетом позиции слушателя и без нее. Слушателем выступает отдельный компонент, определяющий точку в пространстве сцены, относительно которой формируется итоговый звуковой сигнал. Unity допускает только один экземпляр этого компонента на сцене, поэтому в разрабатываемом приложении он должен быть размещен на объекте камеры в тех сценах, где предусмотрен пространственный режим воспроизведения звука.

Большинство источников звука должны быть представлены отдельными компонентами на соответствующих игровых объектах и не предусматривать дополнительной логики управления.

Для звуков окружения, на определенных участках уровня, необходимо предусмотреть аудио-зоны, меняющие способ воспроизведения звука при отсутствии или нахождении в ней одного или обоих персонажей. Если персонажей в аудио-зоне нет, звук не должен быть слышен, если персонаж один – источник звука должен следовать за ним и переключиться в пространственный режим воспроизведения. Если персонажа в аудио-зоне два, необходимо перевести источник звука в непространственный режим воспроизведения.

Звуковое сопровождение взаимодействия с пользовательским интерфейсом должно воспроизводиться без учета положения элемента интерфейса в пространстве.

2.5.11 Система локализации

Система локализации должна обеспечивать загрузку и отображение текстового содержимого игры на языке, выбранном игроком, без необходимости перезапуска приложения.

Для удобства редактирования текста текстовые данные должны храниться в таблице, где первая колонка может содержать строковый ключ, а каждая последующая – перевод для соответствующего языка.

При запуске приложения компонент загрузки локализации, должен обрабатывать файл с таблицей локализации и составлять из него словарь. Определение языка по умолчанию должно происходить автоматически – система должна запрашивать текущий язык через API Steam и использовать его, а если данный язык не входит в число поддерживаемых необходимо по умолчанию необходимо использовать английский язык.

Все текстовые компоненты в сцене должны реагировать на изменение языка и обновлять свое содержимое. Если перевод по заданному ключу языка отсутствует, система должна возвращать английский вариант в качестве запасного.

Поскольку из-за специальных символов и особенностей написания для разных языков требуются разные шрифты, поддержка этих шрифтов должна реализовываться через реестр, сопоставляющий код языка с соответствующими ресурсами шрифта.

2.5.12 Система сохранения и загрузки данных

Система сохранения и загрузки данных отвечает за запись, чтение и удаление игрового прогресса и настроек пользователя. При запуске приложения система должна определять, доступно ли облачное сохранение через Steam Cloud, и если оно актуальнее локального, загружать данные из него. Загруженные данные должны быть доступны другим системам приложения на протяжении всей игровой сессии.

При проектировании данной системы был применен паттерн «одиночка», гарантирующий существование единственного экземпляра класса с глобальной точкой доступа к нему. Этот паттерн нередко рассматривают как антипаттерн из-за введения глобального состояния, однако для системы сохранений его использование обосновано тем, что она должна существовать в единственном экземпляре и оставаться доступной из любой

точки приложения в течение всей игровой сессии. Для наглядного представления структуры была разработана UML-диаграмма классов системы сохранений, представленная на рисунке 11.

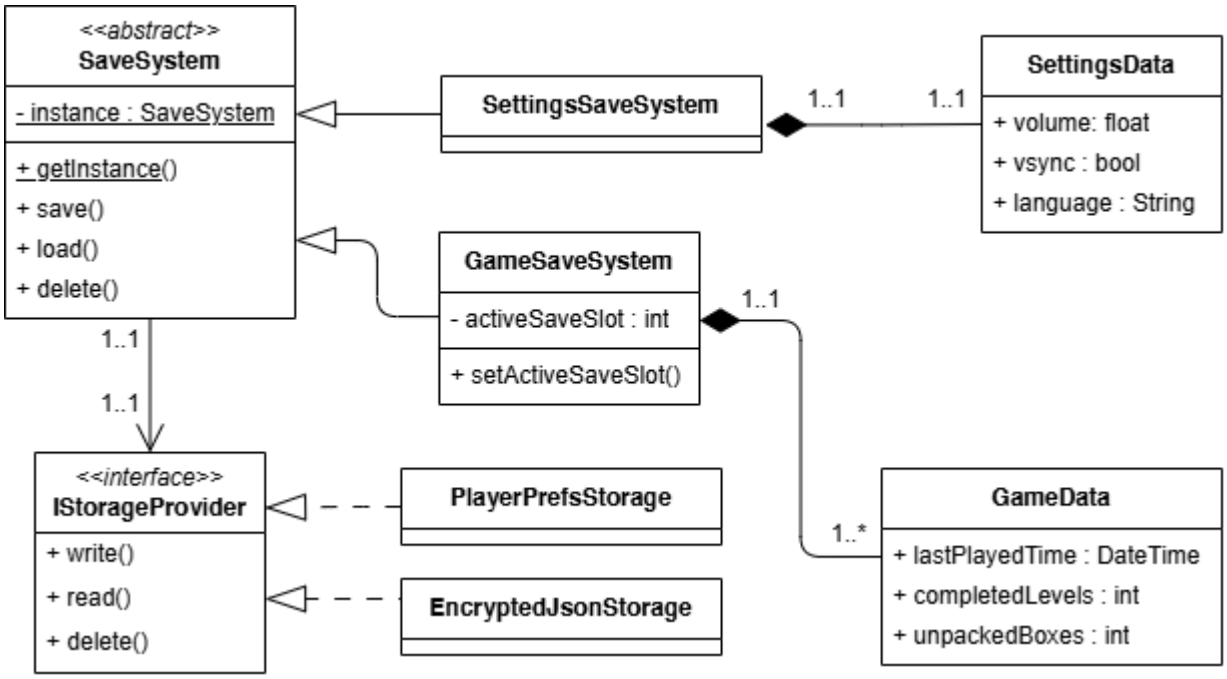

Рисунок 11 – UML-диаграмма классов системы сохранений

Центральным элементом системы является абстрактный класс SaveSystem, определяющий общий интерфейс сохранения, загрузки и удаления данных. Данный класс хранит единственный экземпляр самого себя в статическом поле instance и предоставляет к нему глобальную точку доступа.

На основе SaveSystem спроектированы два конкретных класса – GameSaveSystem для игрового прогресса и SettingsSaveSystem для пользовательских настроек, каждый из которых связан со своим классом данных, атрибуты которого описывают сохраняемое состояние.

Запись и чтение данных делегируются через интерфейс IStorageProvider, позволяющий подменять механизм хранения независимо от остальной логики. Так, на основе данного интерфейса спроектированы два варианта его реализации – PlayerPrefsStorage, использующая встроенный механизм хранения данных Unity, и EncryptedJsonStorage, сохраняющая данные в зашифрованном JSON-файле.

2.5.13 Интеграция с платформой Steam

Программный продукт предназначен для публикации на цифровой платформе Steam, для взаимодействия с сервисами которой была выбрана библиотека Steamworks.NET. Данная библиотека написана на языке программирования C#, совместима с Unity и повторяет структуру вызовов оригинального C++ API от Valve [26].

Посредством взаимодействия со Steamworks.NET приложение должно проверять лицензию при запуске, фиксировать достижения игрока и синхронизировать сохранения с облачным хранилищем Steam Cloud.

3 Разработка компьютерной игры «Cats Together»

3.1 Структура проекта и организация процесса разработки

3.2 Реализация программного продукта

3.2.1 Игровые сцены и уровни

3.2.2 Обработка пользовательского ввода

3.2.3 Управление персонажами

3.2.4 Управление камерой

3.2.5 Взаимодействие с интерактивными объектами

3.2.6 NPC и диалоги

3.2.7 Управление паузой

3.2.8 Пользовательский интерфейс

3.2.9 Пользовательские настройки

3.2.10 Воспроизведение аудио и интеграция звуковых ресурсов

3.2.11 Локализация

3.2.12 Сохранение и загрузка данных

3.2.13 Интеграция с платформой Steam

3.3 Тестирование разработанного программного продукта

ЗАКЛЮЧЕНИЕ

Разработанный программный продукт успешно внедрен в деятельность заказчика и размещен на цифровой платформе Steam, что подтверждается актом о внедрении, представленным в приложении А.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Игры с кооперативными режимами принесли 8,2 миллиарда долларов валового дохода в Steam в 2025 году. – URL: https://www.notebookcheck-ru.com/Igry-s-kooperativnymi-rezhimami-prinesli-8-2-milliarda-dollarov-valovogo-dokhoda-v-Steam-v-2025-godu.1218510.0.html (дата обращения 20.02.2026).
2. Salen K., Zimmerman E. Rules of Play: Game Design Fundamentals. – Cambridge: MIT Press, 2004. – 670 p.
3. Egenfeldt-Nielsen S., Smith J. H., Tosca S. P. Understanding Video Games: The Essential Introduction. – New York: Routledge, 2024. – 400 p.
4. From Solo to Social: Exploring the Dynamics of Player Cooperation in a Co-located Cooperative Exergame. – URL: https://arxiv.org/abs/2504.00160 (дата обращения 20.02.2026).
5. Грис С. Ошибки разработчиков видеоигр. От идеи до провала. – М.: АСТ, 2023. – 320 с.
6. Fullerton T. Game Design Workshop: A Playcentric Approach to Creating Innovative Games. – Boca Raton: CRC Press, 2024. – 414 p.
7. Грегори Д. Игровой движок. Программирование и внутреннее устройство. Третье издание. – СПб.: Питер, 2024. – 1136 с.
8. Baron D. Game Development Patterns with Unity 2021. Second Edition. – Birmingham: Packt Publishing, 2021. – 220 p.
9. Fireboy & Watergirl. – URL: https://chace-dream-company.fandom.com/wiki/Fireboy_%26_Watergirl (дата обращения 01.03.2026).
10. Playing with friends equals gaming success. – URL: https://mcvuk.com/business-news/pc/playing-with-friends-equals-gaming-success/ (дата обращения 01.03.2026).
11. Fireboy and Watergirl: Differences from game to game. – URL: https://www.coolmathgames.com/blog/fireboy-and-watergirl-differences-from-game-to-game/ (дата обращения 01.03.2026).

12. Split Fiction. – URL: https://www.hazelight.se/games/split-fiction/ (дата обращения 10.03.2026).

13. Split Fiction: Josef Fares interview. – URL: https://news.xbox.com/en-us/2025/03/05/split-fiction-josef-fares-interview/ (дата обращения 10.03.2026).

14. Unravel Two. – URL: https://www.ea.com/games/unravel/unravel-two (дата обращения 01.03.2026).

15. Unravel Two Review (Switch). – URL: https://www.nintendoworldreport.com/review/50134/unravel-two-switch/ (дата обращения 01.03.2026).

16. BattleBlock Theater. – URL: https://www.battleblocktheater.com/game/ (дата обращения 09.03.2026).

17. BattleBlock Theater Overview. – URL: https://ryanhorn.desi/battleblock-theater/ (дата обращения 09.03.2026).

18. Unity Plans & Pricing. – URL: https://unity.com/products (дата обращения 24.04.2026).

19. Unity Supported Platforms. – URL: https://support.unity.com/hc/en-us/articles/206336795-What-platforms-are-supported-by-Unity (дата обращения 24.04.2026).

20. Unreal Engine. – URL: https://www.unrealengine.com/ (дата обращения 24.04.2026).

21. Godot Engine Features. – URL: https://godotengine.org/features/ (дата обращения 24.04.2026).

22. GameMaker. – URL: https://gamemaker.io/en (дата обращения 24.04.2026).

23. Microsoft Visual Studio. – URL: https://visualstudio.microsoft.com/vs/ (дата обращения 27.04.2026).

24. JetBrains Rider. – URL: https://www.jetbrains.com/rider/ (дата обращения 27.04.2026).

25. JetBrains Rider для Unity. – URL: https://www.jetbrains.com/lp/dotnet-unity/ (дата обращения 27.04.2026).

26. Steamworks.NET. – URL: https://steamworks.github.io/ (дата обращения 08.05.2026).

ПРИЛОЖЕНИЕ А

Акт о внедрении

АКТ О ВНЕДРЕНИИ

Результатов выпускной квалификационной работы

Кондратовой Екатерины Алексеевны

Настоящий акт составлен о том, что программный продукт, разработанный в ходе выполнения выпускной квалификационной работы студента Балтийского государственного технического университета «ВОЕНМЕХ» им. Д. Ф. Устинова Кондратовой Екатерины Алексеевны на тему «Разработка компьютерной игры «Cats Together» внедрен в деятельность индивидуального предпринимателя Семочкина Ивана Сергеевича и размещен на цифровой платформе Steam.

Индивидуальный предприниматель _____ Семочкин И. С.

«27» сентября 2025 г.

М.П.

ИП Семочкин Иван Сергеевич
194356, Санкт-Петербург, пр. Луначарского, д. 5, корп.1
тел. +7 (999) 232-88-84
E-mail: joraalzak@yandex.ru
ОГРНИП: 321784700266554
ИНН: 780219755362

ЧЕРНОВИК